

Submit VI

Diseño de Algoritmos Paralelos

Juan Diego Collazos Mejia
Juan Sebastian Garizao Puerto

Parallel Maximum and Minimum

 `parallel-maximum-and-minimum.cpp` in GitHub

1. Descripción general del problema

El objetivo de este algoritmo es determinar el valor máximo y el valor mínimo de un conjunto de números almacenados en un vector. En una implementación secuencial, se recorre todo el arreglo comparando cada elemento con los valores actuales de máximo y mínimo, con una complejidad temporal de $O(N)$.

El diseño paralelo busca reducir el tiempo total de ejecución dividiendo el vector entre varios hilos (*threads*) que trabajan de manera simultánea.

2. Método de paralelización

El método utilizado se basa en el patrón de **reducción paralela**:

- Se divide el vector de entrada en N_THREAD segmentos.
- Cada hilo calcula localmente el valor máximo y mínimo de su segmento (**reducción local**).
- Los resultados parciales se combinan (**reducción global**) para obtener los valores finales.

Este enfoque permite explotar el paralelismo de datos, ya que las operaciones de comparación son independientes entre hilos.

3. Descripción del algoritmo secuencial

Entrada: Un vector A de tamaño n .

Salida: El valor máximo y mínimo del conjunto.

```
int max_val = -INF;  
int min_val = INF;
```

```

for (int i = 0; i < n; ++i) {
    max_val = max(max_val, A[i]);
    min_val = min(min_val, A[i]);
}

```

Complejidad temporal: $O(n)$ Complejidad espacial: $O(1)$

4. Diseño del algoritmo paralelo

Idea principal:

1. Crear T hilos (donde $T \leq N_THREAD$).
2. Asignar a cada hilo un rango $[start, end)$ del vector.
3. Cada hilo calcula su máximo y mínimo local.
4. El hilo principal espera la finalización de todos los hilos y combina los resultados locales.

Estructura de datos:

- `struct tdata`: contiene el rango asignado y los valores locales.
- `pthread_t threads[T]`: arreglo de identificadores de hilos.

5. Pseudocódigo del algoritmo paralelo

```

procedure PARALLEL_MAX_MIN(A, N, N_THREAD)
    divide A en N_THREAD segmentos
    for each thread i do
        start_i <- inicio del segmento
        end_i <- fin del segmento
        crear hilo con funci n PARTIAL_MAX_MIN(A, start_i, end_i
        )
    end for

    esperar a que todos los hilos terminen
    global_max <- -INF
    global_min <- INF
    for each hilo i do
        global_max <- max(global_max, local_max[i])
        global_min <- min(global_min, local_min[i])
    end for

    return (global_max, global_min)
end procedure

```

6. Función local de cada hilo

```

procedure PARTIAL_MAX_MIN(A, start, end)
  local_max <- -INF
  local_min <- INF
  for i = start to end do
    local_max <- max(local_max, A[i])
    local_min <- min(local_min, A[i])
  end for
  return (local_max, local_min)
end procedure

```

7. Sincronización y comunicación

Cada hilo trabaja de forma independiente sobre una porción de datos, por lo que no requiere sincronización interna. La única sincronización ocurre cuando el hilo principal (*main thread*):

- Espera a que todos los hilos finalicen usando `pthread_join`.
- Combina los resultados locales.

Por tanto, la reducción global requiere $\lceil \log_2(T) \rceil$ pasos de combinación.

8. Complejidad y análisis

- **Tiempo secuencial:** $O(n)$
- **Tiempo paralelo:** $O(n/T) + O(\log T)$
- **Speedup:** $S = T/(1 + \alpha(T - 1))$, donde α es la fracción secuencial.
- **Eficiencia:** $E = S/T$

9. Resultados esperados

La paralelización reduce el tiempo de ejecución para vectores grandes. Para un número de hilos T pequeño (e.g., 2–4), el tiempo se reduce casi linealmente, pero el beneficio disminuye al aumentar T debido al costo de sincronización y creación de hilos.

Diseño de Algoritmo Paralelo: Prefix Sum (Scan)

 `parallel-scans-and-prefix-sum.cpp` in GitHub

1. Descripción del problema

El cálculo del **Prefix Sum** (también llamado *scan*) consiste en transformar un vector de entrada $A = [a_0, a_1, \dots, a_{n-1}]$ en un vector de salida $P = [p_0, p_1, \dots, p_{n-1}]$, donde cada elemento es la suma acumulada de los elementos anteriores:

$$p_i = \sum_{j=0}^i a_j, \quad \text{para } 0 \leq i < n$$

Por ejemplo, para $A = [2, 4, 1, 7, 3, 0, 4, 2]$, el resultado es $P = [2, 6, 7, 14, 17, 17, 21, 23]$.

La implementación secuencial tiene una complejidad temporal de $O(n)$, ya que requiere recorrer todo el vector. El objetivo del diseño paralelo es reducir el tiempo total mediante la división del trabajo entre varios hilos que operan de forma simultánea.

2. Método de paralelización

El algoritmo se basa en un patrón de **reducción en árbol binario** (*binary tree reduction*), el cual permite que los cálculos se realicen en fases independientes y paralelas. Consta de dos fases principales:

1. **Upsweep (reducción)**: se construye un árbol de sumas parciales. En cada paso, la mitad de los hilos combina pares de elementos separados por una distancia creciente (2^d). Al finalizar, el último elemento contiene la suma total.
2. **Downsweep (distribución)**: la suma total se propaga de nuevo hacia los niveles inferiores del árbol. En cada paso, los valores parciales se distribuyen hacia la izquierda, produciendo los prefijos acumulados.

Cada nivel del árbol puede ejecutarse en paralelo, ya que las operaciones de suma entre pares no tienen dependencias entre sí. El número de pasos de sincronización requeridos es:

$$\text{Pasos de sincronización} = \lceil \log_2(n) \rceil$$

3. Descripción del algoritmo secuencial

Entrada: Vector A de tamaño n .

Salida: Vector P de prefijos acumulados.

```
P[0] = A[0];
for (int i = 1; i < n; i++) {
    P[i] = P[i-1] + A[i];
}
```

4. Diseño del algoritmo paralelo con pthread

Idea general:

1. Dividir el vector en segmentos procesados por distintos hilos.
2. Cada hilo participa en las fases de **upsweep** y **downsweep**, sumando pares de valores en pasos de tamaño creciente.
3. Entre cada paso se utiliza una **barrier** de sincronización para asegurar que todos los hilos completan el nivel antes de pasar al siguiente.

5. Pseudocódigo paralelo

```
procedure PARALLEL_PREFIX_SUM(A, n, T)
    barrier_init(T)

    // UPSWEEP
    for d = 0 to log2(n)-1 do
        parallel for i = 0 to n-1 step 2^(d+1) do
            A[i + 2^(d+1) - 1] = A[i + 2^d - 1] + A[i + 2^(d+1) - 1]
        end for
        barrier_wait()
    end for

    if thread_id == 0 then
        A[n-1] = 0
        barrier_wait()

    // DOWNSWEEP
    for d = log2(n)-1 downto 0 do
        parallel for i = 0 to n-1 step 2^(d+1) do
            t = A[i + 2^d - 1]
            A[i + 2^d - 1] = A[i + 2^(d+1) - 1]
            A[i + 2^(d+1) - 1] = t + A[i + 2^(d+1) - 1]
        end for
```

```

        barrier_wait()
    end for
end procedure

```

6. Sincronización y comunicación

Cada hilo trabaja en una porción distinta del vector, por lo que no existen dependencias directas entre ellos dentro del mismo nivel. Sin embargo, es necesario garantizar que todos los hilos terminen un nivel antes de iniciar el siguiente, para lo cual se emplea una **barrier** global.

- Fase **Upsweep**: requiere $\lceil \log_2(n) \rceil$ pasos.
- Fase **Downsweep**: requiere también $\lceil \log_2(n) \rceil$ pasos.

Por lo tanto, el número total de sincronizaciones es $2 \cdot \lceil \log_2(n) \rceil$.

7. Complejidad y eficiencia

- **Tiempo secuencial**: $O(n)$
- **Tiempo paralelo**: $O(\log n)$ por hilo
- **Speedup**: $S \approx \frac{n}{\log n}$
- **Eficiencia**: $E = \frac{S}{T}$

El rendimiento mejora considerablemente para tamaños grandes de n , aunque se ve afectado por el costo de sincronización cuando se utilizan muchos hilos.